ParaThink: Speech Frontend, Parallel Thinking and Expression

Continuous observation / Shared context / One playback owner

Incoming user audio

Speech Frontend: Native SoulX Control + Independent X2 Text

Independent X2 Streaming ASR
Two native 80 ms audio steps
LISTEN / DECODE to real EOS

Same continuous PCM
Independent text / control

Native SoulX + SenseVoiceSmall
Internal ASR / rules / feedback / reset
Five native user states

Each 160 ms: finish both lanes, atomically publish, then advance. WAIT also completes a step.

Future CoT input

One atomic 160 ms commit: text + states + progress

Shared Context: Available Dialogue + Runtime State

Current EOS-closed text + five states + real progress / One joint batch snapshot / Layer 2 CoT
Layer 1 speech and backchannel output / Listen–Speak state
Interruption events / generated, submitted, canceled and played records

Read

Output

Read

CoT

Layer 1
Expression
Read the joint snapshot
Choose backchannel timing and words
Reply after committed complete
Request thinking

Request

Layer 2
Async Thinking
Publish CoT only
No turn-action advice
Update one active task

Frontend observation, expression and thinking can run concurrently.

Actual Listen / Speak state + interruption events

Speech / backchannel text

Text FIFO

Submitted speech text / system backchannel, in order
Chunks carry output_id + chunk_id; accept active output only

Committed nonidle
Match output ID
Stop once

Consume in order

Output Runtime: Streaming TTS + Player

Single arbitration point / single owner of audible output
Priority stop: halt audio + cancel pending output
Reject late text / audio carrying an invalid output_id

Audio heard by the user: never revoked

On a valid stop: execution in the same output runtime

1. Close old output_id
Block stale submissions

2. Halt + cancel
Cancel queued / buffered output
Discard late text / audio

3. Closure acknowledged
Publish Listen + interruption
Keep generated history

4. New Layer 1 reply
Read committed complete + text
Fresh output_id; new text

Five states share one commit boundary. Committed valid complete can be used with the current text.

Native frontend validation is recorded. Downstream playback and CoT integration remain pending.